Programação I

Prof. Maurício M. Arimoto / Prof. Paulo R. Anastácio

Aluno: _____ **Matrícula:** _____

Data: 05.08.2026

1. Indique quais as saídas produzidas pelo programa a seguir. Faça o teste de mesa de cada instrução e verifique os resultados. Depois, você pode executar o código comparando os resultados.

```c
#include <stdio.h>

int main() {
    int vet1[] = {1, 2, 3, 4, 5, 6, 7};
    int vet2[] = {7, 6, 5, 4, 3, 2, 1};
    int *ptr1 = vet1;
    int *ptr2 = vet1 + 3;
    int *ptr3 = vet2 + 5;
    (*ptr1)++;
    (*ptr2)++;
    (*ptr3)--;
    printf("vet1[0]: %d, vet1[3]: %d,  vet2[5]: %d\n", vet1[0], vet1
        [3], vet2[5]);
    return 0;
}
```

2. Analise a sequência de instruções a seguir:

```c
int x = 10, y = 5;
int *ptr1 = &x;
int *ptr2 = &y;
```

Quais expressões abaixo são válidas e quais não são válidas? Justifique sua resposta.

(a) y = ptr1 == ptr2;

(b) ptr1 += ptr2;

(c) x = (*ptr1) - (*ptr2);

(d) x = ptr1 || ptr2;

(e) (*ptr2)++;

(f) y = ptr1 && ptr2;

3. Indique quais as saídas produzidas pelo programa a seguir. Faça o teste de mesa de cada instrução e verifique os resultados. Depois, você pode executar o código comparando os resultados.

```c
#include <stdio.h>

int main() {
    int x = 5, y = 7;
    int *px = &x;
    int *py = &y;
    int **ppx = &px;
    int **ppy = &py;
    int ***pppx = &ppx;
    int ***pppy = &ppy;

    ppx = ppy;
    x = **ppy;
    ***pppy += (**ppx);

    printf("%d %d \n", x, y);
    printf("%d %d %d \n", ***pppx, ***pppy, px != py);
    return 0;
}
```

4. Indique quais as saídas produzidas pelo programa a seguir. Faça o teste de mesa de cada instrução e verifique os resultados. Depois, você pode executar o código comparando os resultados.

```c
#include <stdio.h>

int main() {
    int matriz[2][2] = {{1, 3}, {5, 7}};
    int *p1 = &matriz[0][0];
    int *p2 = &matriz[1][0];
    int **pp1 = &p1;
    int **pp2 = &p2;
    int ***ppp1 = &pp1;

    ***ppp1 = 2;
    *(*pp1 + 2) = 4;
    *(*pp2 + 1) = 9;

    printf("%d %d\n", matriz[0][0], matriz[0][1]);
    printf("%d %d\n", matriz[1][0], matriz[1][1]);

    return 0;
}
```

5. Considere o trecho de código a seguir:

```c
int a = 20, b = 15;
int *p1 = &a;
```

```
3 int *p2 = &b;
4 int **pp1 = &p1;
5 int **pp2 = &p2;
```

Para cada expressão independente, indique se é válida ou inválida em C (considerando compilação e/ou comportamento) e justifique sua resposta:

(a) *pp1 = *pp2;

(b) **pp1 = (*p2 += **pp1);

(c) b = (pp1 == &p1) + (*pp1 == &a);

(d) p1 = pp2;

(e) *pp1 = (p1 && p2);

(f) a = (*pp1 == *pp2) ? **pp2 : **pp1;

6. Utilizando aritmética de ponteiros, mostre como exibir a frase "não gosto de programar em C"como "gosto de programar em C".

7. Faça um programa que leia uma matriz quadrada de ordem 4 X 4 de números inteiros usando ponteiros. Depois, leia um número x e verifique quantas vezes x aparece na matriz.

8. O coordenador do curso deseja saber quantos alunos estão cursando ao mesmo tempo as disciplinas de Programação I e Programação II. Faça um programa usando ponteiros que leia os códigos de matrícula dos alunos de ambos os cursos e mostre os códigos de matrículas dos alunos que estão cursando ambas as disciplinas.

9. Reescreva o programa abaixo usando ponteiros.

```c
#include <stdio.h>
#define MAX 255

int main() {
    char str[MAX], caractere;
    int count = 0;

    printf("Entre com a string: ");
    fgets(str, MAX, stdin);
    printf("Entre com o caractere: ");
    scanf(" %c", &caractere);

    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] == caractere) {
            printf("%d\n", i);
            count++;
        }
    }
    if (count == 0)
        printf("-1\n");
    return 0;
}
```